

# Canada Guesser

## COMP 3106A - Introduction to Artificial Intelligence

Authors: Stephen Rebel (XXXXXXXXXX) - Danial McIntyre (XXXXXXXXXX) - Sharav Bali (XXXXXXXXXX)

Date: December 5, 2025

## Statement of Contributions

All members have made significant and meaningful contributions to the project planning, implementation, and reporting. Over the course of this project team members have made equal contributions across all sections. All three members were involved with the planning of the project. Stephen and Danial worked on the data curation script together. Stephen designed the convolutional neural network (CNN) validation script, and Danial designed the validation script for vision transformers. Stephen developed a result visualization script for validation with some inputs for final training from Sharav. Sharav primarily designed the CNN final training script, and Danial worked on vision transformer final training. Sharav and Danial also both contributed to the final demonstration script. Stephen contributed to the introduction and methods sections primarily with contributions from Danial and Sharav on their methods. Sharav completed the results section. All members contributed to the discussion and document revision.

## Introduction

### Background and Objectives

Geographical and locational awareness can be a very valuable skill in both recreational and practical situations. The ability to identify one's location based on visual cues from the surrounding environment is the motivation behind this study. The inspirations for this work come from two sources: the video game GeoGuesser, and its applicability to real-world situations. GeoGuesser is a game where a player is placed in a random geographic location in the world and must guess where they are based on panoramic streetview surroundings (GeoGuesser AB, 2013). This game can help develop skills that can become very practical in real-world scenarios supposing one does not have access to other geopositioning devices and needs to roughly determine where they are. Our interest lies in Canadian geography and locations, as such our domain and objective is focused on the prediction of locations across 15 major Canadian cities, spanning the east coast to the west coast. Canada is often underrepresented in large scale training data and AI research. To this extent we curate a custom dataset focused on Canada and develop machine learning models capable of predicting a location with a high degree of accuracy given a streetview image. Such a model can be used in practical scenarios empowering users who do not have the ability to predict their location given surrounding cues.

### Related Prior Work

Many prior studies and research efforts were focused on global location prediction or specific countries such as the United States. In *DeepGeo*, Suresh et al. (2018) framed image geolocation as a classification problem. To accomplish their goal Suresh et al. (2018) designed a ResNet Convolutional Neural Network (CNN) (He et al., 2015) varying the stage of the model at which input images from different cardinal directions of a location are integrated. *PlaNet* (Weyand et al., 2016) is a widely cited study which also introduced the problem of image geolocation as a classification problem. Weyand et al. (2016) represented the task of global image geolocation as classification to tiles over the globe, employing deep CNNs based on the Inception architecture, formerly known as *GoogLeNet* (Szegedy et al., 2014). Employing CNNs as classifiers to predict image geolocation at a city granularity level is highly effective as seen in results from Vo et al. (2017) using the IM2GPS dataset. Features extracted by CNNs remain effective as remarked by Vo et al. (2017) and strong even with limited data, and can effectively be used for various downstream tasks beyond classification.

In the past image classification has been dominated by CNN architectures yet recently vision transformers have been highly successful on image classification tasks among others. When they were first invented, transformers were mostly used for natural language processing, as they can be trained on a large text base, then finetuned for a more specific task (Vaswani et al., 2017). The most famous example that has gone viral within the past few years is ChatGPT, where the GPT stands for Generative Pre-Trained Transformer (Introducing ChatGPT, 2022). It was not until 2021 that transformers were first used for image classification with the introduction of the vision transformer, or ViT (Dosovitskiy et al., 2021), and outperformed the state-of-the-art CNNs of the time. Since then, the use of vision transformers for classifying images has increased, with Xie et al. (2024) comparing the SwinV2 transformer to DenseNet-169 CNN to determine the walkability of street view images. They concluded that vision transformers have better feature recognition than CNNs. We wanted to explore whether these findings could be applied to a previously untested domain: classifying cities in Canada from street view images.

Previous works have obtained data from multiple sources. In *DeepGeo*, Suresh et al. (2018) used the Google Street View API. In the wake of Google’s restrictions upon their street view images and other data sources for use in machine learning tasks, many studies such as Astruc et al. (2024), have used alternative sources. Namely Mapillary (Mapillary AB, 2013), an openly crowd-sourced street view platform. Using the Mapillary API, Astruc et al. (2024) created 100m by 100m tiles across the globe and randomly sampled one image per tile. *PlaNet* (Weyand et al., 2016) and *IM2GPS* (Hays & Efros, 2008) both web scrape images with geolocation tags in addition to FLICKR (SmugMug, Inc., 2004) images. This approach however, suffers from collecting all forms of images including those of indoor scenes or images of pets as long as they have geolocation metadata.

## Methods

### Methods from Artificial Intelligence

Previous works primarily used CNN architectures as a means for classification, and we aimed to test their abilities during our validations. In addition, more recent methods note the power of vision transformers, so we elected to test out a few models ourselves. We selected a few modern CNN architectures recently released which have not been tested by any of the previous studies. Among the many CNN architectures, we decided to validate a few of varying sizes, thus selecting EfficientNetV2 small (Tan & Le, 2021), ConvNeXt tiny (Liu et al., 2022), and EfficientNet B6 (Tan & Le, 2020). The vision transformer architectures we selected for our experiments include ViT base (Dosovitskiy et al., 2021), DeiT 3 (Touvron et al., 2022), and SwinV2 (Liu et al., 2022). The rationale was to curate a subset of models and architectures representative of current state-of-the-art vision models and apply them anew to this task, and to discover which model size or architecture achieved the best performance on this constrained geolocation prediction task.

To train highly effective vision models requires training the vision head over millions of data samples to enable learning, and extracting meaningful features from diverse images. One popular pretraining approach is to employ the ImageNet dataset (Russakovsky et al., 2015). This pretraining however, is infeasible for many research efforts due to resource constraints, as such we sourced pretrained vision model weights from the PyTorch Image Models repository (Wightman, 2019). This enabled us to employ transfer learning where the final fully connected layers were replaced with a new classification head. For our CNN training, the model backbone was frozen while the classifier alone was trained for some number of epochs before unfreezing the entire model for continued learning and fine tuning over the feature extraction backbone. While for transformer training no freezing was employed and all parameters were trained simultaneously. Epochs and learning rates varied based on validation sets and will be discussed in greater detail subsequently.

Training was conducted using an Nvidia 3060 Ti GPU with 8GB of VRAM using PyTorch as our training engine. Torch AutoCast was employed to make efficient use of resources, thus training was conducted using bfloat16 quantization and float32 as determined by AutoCast. During training we employed the standard cross entropy loss function with label smoothing defined by the hyperparameters. We also selected the AdamW optimizer to enable the use of weight decay and reduce overfitting.

## Dataset

One of the primary objectives and sources of novelty, was the creation of a custom dataset of streetview images from Canadian cities, as no such dedicated dataset exists for this purpose. We curated a dataset of images selected from 15 cities, the choices of which were made to include diversity across Canada, as well as including major metropolitan centres in the country. The selected cities can be found in Figure 1. As noted by Astruc et al. (2024), Mapillary can be used as an alternative to Google Street View, due to the limitations imposed upon the use of Google's data. It is against their terms of service to use the Street View API to scrape images and to store them long term or for them to be used in machine learning applications (*Google Maps Platform Terms of Service* | *Google Cloud*, 2025). For each city, we collected 10,000 images as suggested by Suresh et al. (2018), by defining the spatial region of each Canadian city. We employed rectangular bounding boxes encompassing the major urban footprint, from which images were sourced. Initially we investigated officially recognized bounding boxes around selected cities from Natural Resources Canada's Geolocation Service (Natural Resources Canada, 2022). However, upon visual inspection these were unsatisfactory encompassing too much of the surrounding non-urban area and in the case of Toronto a significant portion covered Lake Ontario. We instead opted to manually define bounding boxes ourselves, verifying regions visually.

The Mapillary API (Mapillary AB, n.d.) requires a bounding box from which images will be sampled. The API has a limit on the size of bounding boxes of 0.01 degrees squared, as such we required tiling over our larger bounding boxes which was accomplished through Mercantile (Gillies, 2014). Additionally we filtered images, excluding those captured through 360° cameras. We collected metadata for all images returned per city and saved the request URL for 1024px width images. From the total list of candidates, we sampled and downloaded 10,000 for each city. The full dataset of 150,000 images was partitioned into training (135,000 images) and testing (15,000 images) splits using stratification by city. The final curated dataset was uploaded to Hugging Face (Rebel et al., 2025) contributing to the open-source community, accumulating 150 downloads as of writing.

During the training of our CNNs, images were preprocessed using PyTorch transforms V2 (PyTorch, n.d.). We first applied a letterboxing approach to maintain aspect ratio within a fixed input size of  $320\text{px} \times 320\text{px}$ , vertically centering images on a black background. Additionally, images had a uniformly random horizontal flip and mild colour jittering applied, with brightness and contrast each varied randomly within  $\pm 0.1$  of their original values, followed by normalization during model ingestion. These augmentations aim to improve model generalization and stabilize training.

For our vision transformers, images were resized to fit the constraints of the model they were trained on,  $224 \times 224$  for ViT and DeiT, and  $192 \times 192$  for Swin. For training, images were augmented with horizontal flips, rotations (within 10 degrees), and colour jittering. Augmentations to the training set help the model learn the general features of an image, rather than memorizing specific details, which helps with generalization. Other than resizing and normalization, the test set is left without these augmentations, as they represent the real world, which the model is trying to understand.

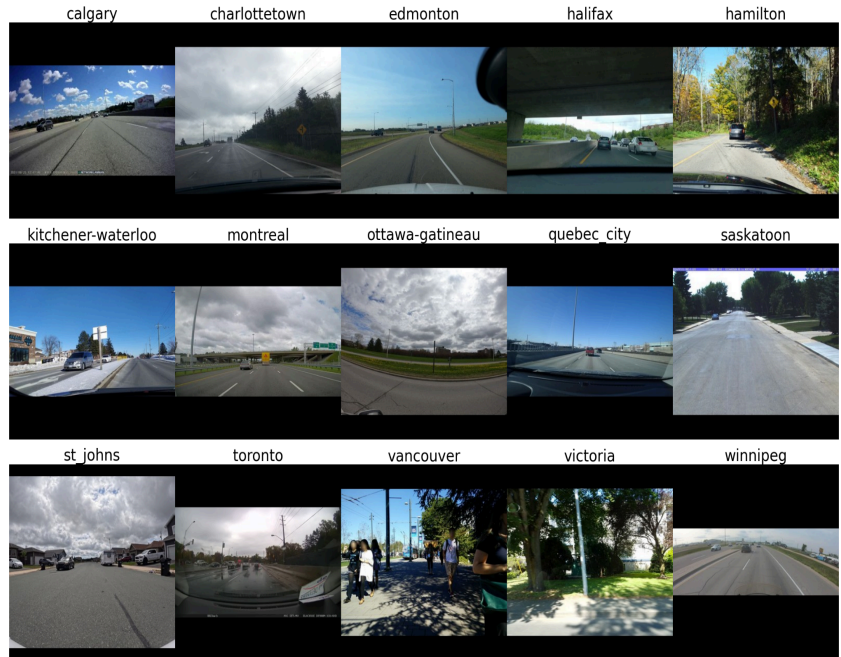


Figure 1: Streetview Image Samples from Canadian Cities

## Validation Strategy and Experiments

We performed an extensive validation across the selected models and a few hyperparameter sets for each model. We employed a 5-fold cross validation procedure where validation occurred over each fold 1-5 (27,000 images), and training occurred over the remaining four folds (108,000 images). Our validation strategy had two goals, first to determine the best performing model architecture (CNN or vision transformer), and second to select the best performing set of hyperparameters for each model. Due to the resource and time constrained nature of our project we defined two to three sets of hyperparameters per model. The hyperparameter sets were generally informed by default parameters of components in the training pipeline. For example we varied the learning rates and weight decay parameters of the AdamW optimizer within a small range of default values. We also varied the number of training epochs and training batch sizes.

Across the 5-fold validation, we collected training and validation loss over the epochs and averaged it across folds to generate loss curves. From these curves we were able to investigate the performance of our models and determine those that were learning well and their rate of convergence. Additionally we collected the highest accuracy predictions a model made during a fold. With the predictions collected we generated a confusion matrix to allow us to analyze where models were having difficulties and where they were performing well. Additionally we computed precision, recall, and F1-score per-class, as well as their macro average and average accuracy across all classes. These metrics permitted us to quantitatively analyze the performance of models and hyperparameter sets, and to select the highest performing combinations for final training. Below are the results from the best performing set/model of both the transformers and CNNs.

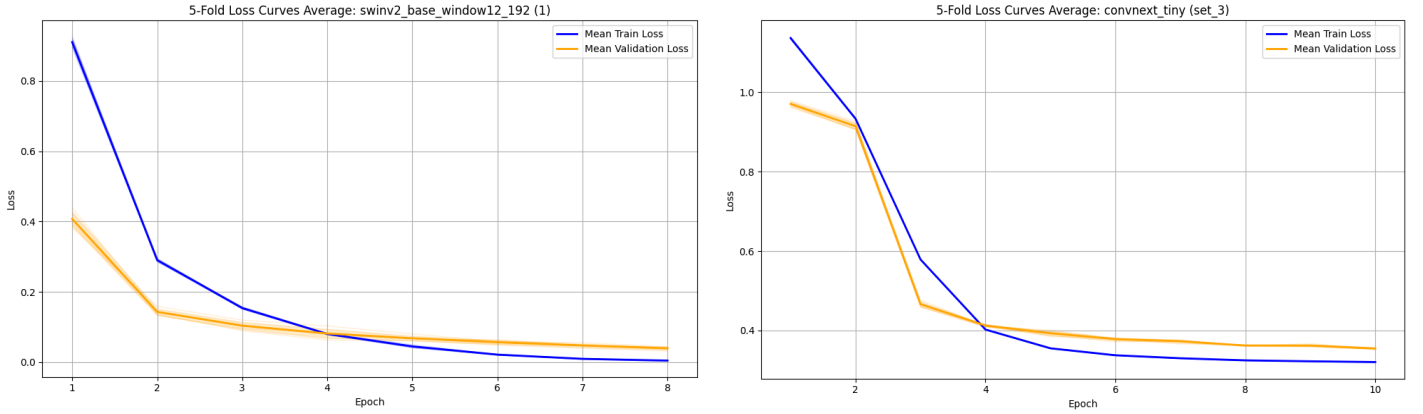


Figure 2: Mean Training and Validation loss of 5-Fold Validation on Swin Transformer V2 Hyperparameter Set 1 (left) and ConvNeXt-tiny Hyperparameter Set 3 (right)

| Model (Hyperparameter)  | Accuracy | Macro Precision | Macro Recall | Macro F1-Score |
|-------------------------|----------|-----------------|--------------|----------------|
| ConvNeXt-tiny (Set 3)   | 0.9885   | 0.9884          | 0.9885       | 0.9884         |
| SwinV2 Base (Set 1)     | 0.9897   | 0.9897          | 0.9897       | 0.9897         |
| DeiT 3 Base (Set 2)     | 0.9875   | 0.9875          | 0.9875       | 0.9875         |
| EfficientNet B2 (Set 1) | 0.9630   | 0.9630          | 0.9630       | 0.9630         |
| ViT Base (Set 1)        | 0.9855   | 0.9855          | 0.9855       | 0.9855         |
| EfficientNetV2 (Set 2)  | 0.9831   | 0.9831          | 0.9831       | 0.9831         |

Table 1: 5-Fold Validation Metrics of Highest Performing Hyperparameter Set for Each Model

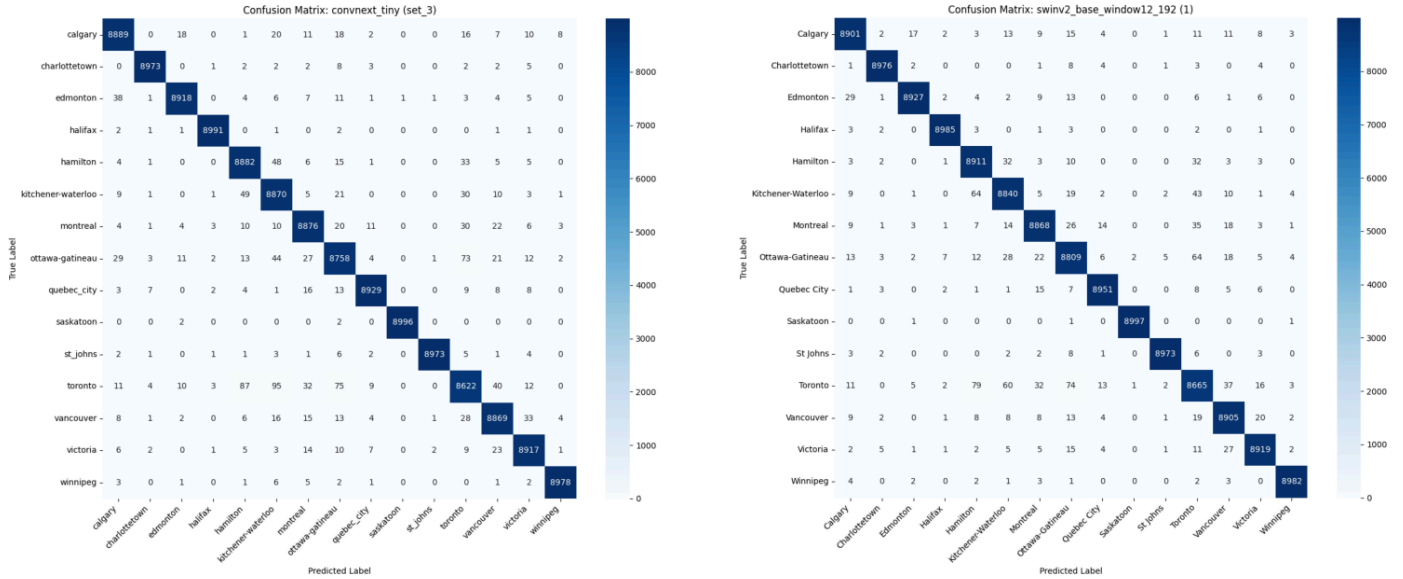


Figure 4: 5-Fold Validation Confusion Matrices of Highest Performing Models

The above figures and tables represent the highest performing CNN and vision transformer that we selected to perform full training on. We made this choice upon careful analysis of all validation data choosing models and hyperparameters that achieved the best accuracy and F1-score while displaying strong learning based on average validation loss. Additionally, confusion matrices compounded these metrics revealing the exact details of what mistakes were committed and for which classes.

## Results

### Qualitative Results

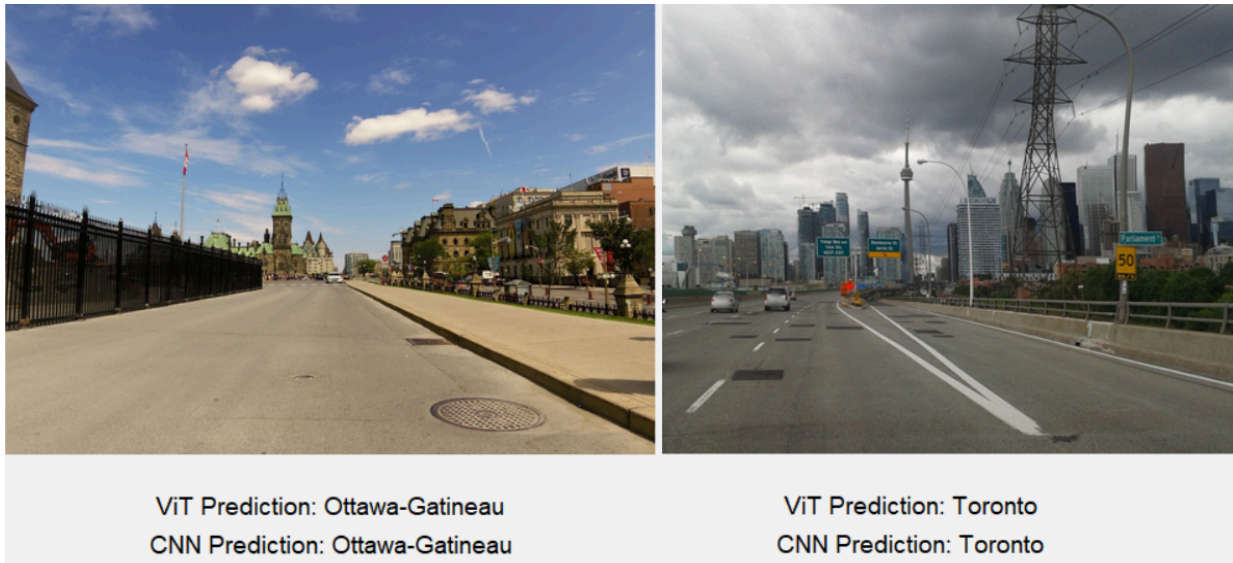


Figure 5: Correct Model Predictions of Ottawa-Gatineau and Toronto

To qualitatively evaluate our model performance, we provided our models with real street-level images from Canadian cities and observed their predicted labels. The above Figure 5 showcases two examples from our interface: one taken near Parliament Hill in Ottawa-Gatineau, and another of the Toronto skyline from an elevated highway. In both cases, our models predicted the correct city. These results demonstrate the models’ abilities to visually interpret locations. The scene from Ottawa-Gatineau contains distinctive landmarks, notably the Parliament buildings, as well as the recognizable fence and road layout leading towards them. These features are strongly correlated with Ottawa-Gatineau in the training dataset, proving it easy for the models to classify the images. The Toronto example illustrates similarly unique qualities: the CN Tower, expressway, as well as the buildings making up the skyline. The recognizability of these elements is effectively captured by both Swin transformer V2 and ConvNeXt-tiny.

In contrast, Figure 6 represents a case in which the transformer model correctly classifies the image as Vancouver, while the CNN incorrectly predicts Toronto. The provided image depicts a scene that lacks a notable landmark and instead contains more subtle cues, such as trolley wires, mid-rise buildings, and trees with a small, visible backdrop. The transformer model is better suited to interpret broader scenes, as it views the scene from a global perspective. The model notices the aforementioned features, and recognizes their spatial distribution, detecting how these elements fit together and understanding that the combination of cues are more common in Vancouver. The CNN, however, relies more heavily on local textures, and building details, which in this case may resemble urban streets found in Toronto, hence the prediction it made. As a result, the CNN model is more easily confused when distinctive texture cues it was trained on are weak, while the transformer shines in a scenario of this nature as it leverages the full context of the landscape.

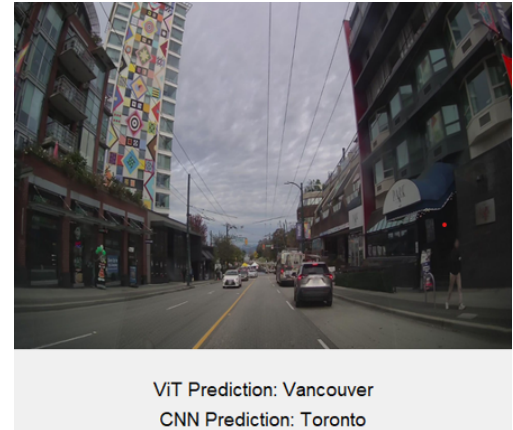


Figure 6: Transformer correctly predicts Vancouver

| Benchmark       | Acc |
|-----------------|-----|
| SwinV2          | 69% |
| ConvNeXt-tiny   | 54% |
| Grok 4          | 54% |
| Human Benchmark | 26% |

Table 2: Different benchmarks tested with 13 out of domain images

We also experimented with images from a different data source, to see how well our model is able to generalize. We found 13 distinct images from Google Street View to test our models with, as well as test with a human benchmark and a top performing LLM. As shown in Table 2, the transformer model performs the best on out of domain images with correctly predicting 69% of images, with the CNN and Grok achieving 54% accuracy. These are all well above what our human benchmarks were able to achieve, showing that these models can be better predictors of cities than humans. Something to note is that this was not an exhaustive experiment, as images were handpicked by us, and the human benchmark we used were some of our friends who volunteered for this task, and were by no means geographic specialists.

## Quantitative Results

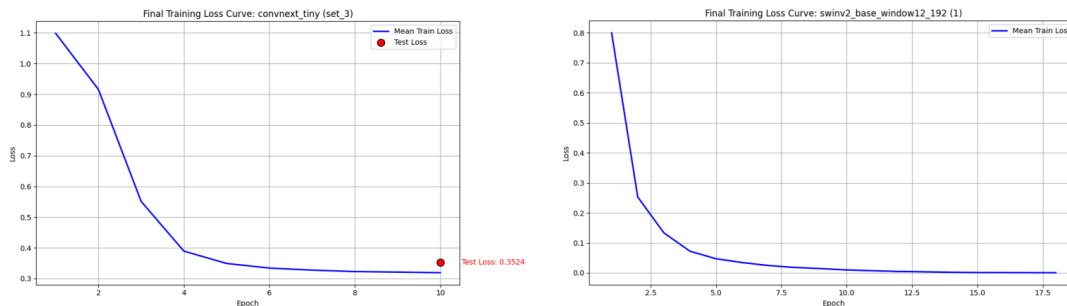


Figure 7: Final Training Loss Curves of ConvNeXt-tiny and Swin Transformer V2

From analyzing the final training loss curves for the CNN model and the transformer model, we observe that both models have stable convergence, showcasing that the training procedures and hyperparameter selections were effective. The CNN’s training loss sharply declines over the first four epochs and then gradually lowers, stabilizing around 0.33. This suggests that the model is not overfitting to a large extent and that our selected hyperparameters from cross-validation generalize unseen images well. The transformer model shows a similar pattern of sharp initial decline followed by smoothing out over the 18 epochs. This indicates to us that our optimization process works well for both models, and is effective in driving these models toward low training loss.

| Model               | Accuracy | Macro Precision | Macro Recall | Macro F1-Score |
|---------------------|----------|-----------------|--------------|----------------|
| ConvNeXt-tiny       | 0.98980  | 0.98983         | 0.98980      | 0.98980        |
| Swin Transformer V2 | 0.99440  | 0.99439         | 0.99440      | 0.99439        |

Table 3: Final Training Metrics of ConvNeXt-tiny and Swin Transformer V2

The final training metrics for both models show very strong performance on the classification task. The accuracy and macro results seen in Table 3 indicate that both models are able to learn greatly discriminative features for distinguishing between the 15 cities in the dataset. Neither model suffers from class imbalance issues either, as we can see with the consistent numbers throughout the table. While the transformer attains slightly higher scores than the CNN, the difference remains miniscule, reflecting the fact that both models converge efficiently when trained on the full dataset.

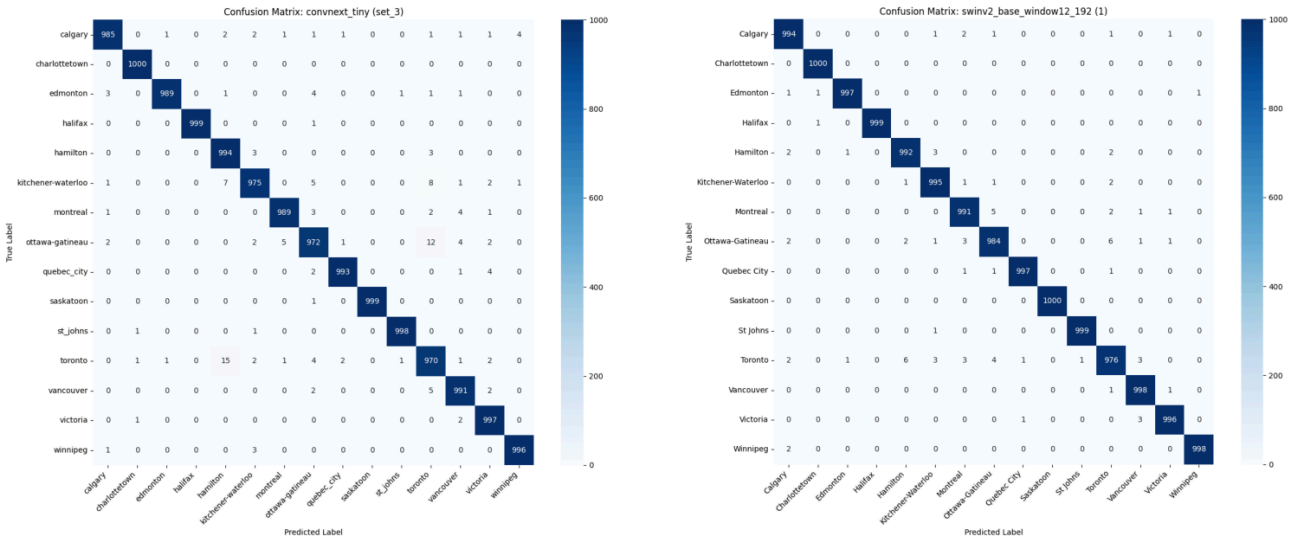


Figure 8: Final Training Confusion Matrices of ConvNeXt-tiny and Swin Transformer V2

The confusion matrices for the CNN and transformer models demonstrate that each model achieves very reliable classification performance across nearly all 15 cities. The diagonal entries are dominant, many of which exceed 990 correct predictions, which is expected as this indicates that both models learn strong city-specific representations. We rarely observe misclassifications, typically occurring among cities with visually similar urban features, such as Montreal, Ottawa-Gatineau, and Toronto, all of which contain metropolitan areas with dense streets, and similar roadway textures. A noteworthy pattern occurs in the Saskatoon row, as both models achieve exceedingly high/perfect accuracy for this class, which compared to the near-perfect diagonal may suggest overfitting. Even with this, the transformer model remains less erratic, reflecting its ability to take advantage of global context to disambiguate cities whose textures may be similar. Meanwhile, the CNN shows occasional confusion hinted at by a lesser accurate performance overall. In summary, both models perform exceptionally well, though subtle differences in our matrices help with proving the broader trend observed in our qualitative results, being that the transformer generalizes more effectively in cases where the CNN may experience confusion due to limiting factors.

# Discussion

## Limitations and Directions for Future Work

The biggest limiting factor that we ran into was finding a good enough dataset for training. While the dataset we were able to curate was good enough for training, we found that it doesn't generalize quite as well to images found in other datasets. This is because some of the images from our dataset have distinct features that the models would memorize, such as a recurring dashcam or dashboard. We found this very problematic for some cities, specifically Saskatoon, as most of the images from the Mapillary database all contain a small red dashcam band at the top of the image. This leads to overfitting, where the models learn the red band, and basically ignore all other content in the image. This was evident by looking at our confusion matrices, where most cities have some values that are wrongly predicted, while Saskatoon would often predict all the test cases perfectly. This problem could be solved with further preprocessing of the images before training, to crop out any features that may lead to overfitting. Another limiting factor is simply the nature of the images found on Mapillary. When testing our models on images from Google Street View, they are nowhere near as accurate as they are with images from Mapillary. We don't know exactly why this is the case, although it is likely because Google Street View images are professionally taken and at a high quality, while Mapillary, being an open source application, contains images of varying quality and locations. Oftentimes for smaller cities, all the images would be submitted by just a handful of contributors, again leading to some form of overfitting. Ideally, we would be able to use a dataset with high quality and reliable images, such as Google Street View, however due to constraints mentioned earlier, this is not possible.

Another constraint we had was with the GPU we were given. With only 8GB of VRAM, we were limited to using the smaller versions of pre-trained models, as well hyperparameters, such as batch size and learning rate. For example, the models we used, `convnext_tiny` and `swinv2_base_window12_192`, have larger versions with more parameters and work with larger sized images. Using these models may lead to superior performance at the cost of additional training.

## Implications of the Work

The results and findings of this project highlight several important implications for the use of deep learning models in the context of geolocation and streetview/urban classification tasks. The project demonstrates that convolutional neural networks and vision transformers are highly capable of noticing important geographical elements from street-level images, most importantly across cities within a range of subtle to major differentiation. Beyond the performance of the project, this work shows the feasibility of applying transfer-learning to a domain where curated datasets are hard to find, motivating further research focused on geolocation in regions that may be underrepresented such as in Canada. Another important implication of the work is providing a foundation from which future researchers can build, with creation of a well-structured open-sourced dataset and models. The dataset can assist with further tasks related to geolocation or street-level classification, or be applied to adjacent tasks. Overall, the work contributes tools to serve practicality as well as methodologies that can support future advancements in artificial intelligence-based geolocation.

## Conclusion

Our objectives were to curate a dataset of Canadian city streetview data and to train deep learning networks capable of identifying the city in which streetview images were captured. We successfully developed and open-sourced the Canadian-streetview-cities dataset (Rebel et al., 2025), consisting of 150,000 images across 15 Canadian cities collected from the Mapillary API (Mapillary AB, n.d.). Using this dataset, we applied a transfer-learning pipeline to evaluate multiple CNN and vision transformer architectures. The final Canadian-streetview-cities-model collection (McIntyre et al., 2025) includes the ConvNeXt-tiny and Swin Transformer V2 models, which achieved high accuracy on our testing set scoring 98.98% and 99.44% respectively. The results demonstrate the effectiveness of our transfer-learning approach and the models exceeded expectations with performance far surpassing that of humans. Overall, the project successfully achieved the objectives of novel dataset creation, model development, and high-accuracy Canadian geolocation, and provided foundations for future researchers interested in Canadian image recognition.

## References

- Astruc, G., Dufour, N., Siglidis, I., Aronsson, C., Bouia, N., Fu, S., Loiseau, R., Nguyen, V. N., Raude, C., Vincent, E., Xu, L., Zhou, H., & Landrieu, L. (2024). *OpenStreetView-5M: The many roads to global visual geolocation*. arXiv. <https://arxiv.org/abs/2404.18873>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). *AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE*. <https://arxiv.org/pdf/2010.11929>
- GeoGuessr AB. (2013). *GeoGuessr* [Video Game]. <https://www.geoguessr.com>
- Gillies, S. (2014). *Mercantile* [Computer software]. GitHub. <https://github.com/mapbox/mercantile>
- Google Maps Platform Terms Of Service | Google Cloud. (2025). Google Cloud. <https://cloud.google.com/maps-platform/terms?hl=en>
- Hays, J., & Efros, A. A. (2008). *IM2GPS: Estimating geographic information from a single image*. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1–8. <https://doi.org/10.1109/CVPR.2008.4587784>
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep residual learning for image recognition*. arXiv. <https://arxiv.org/abs/1512.03385>
- Introducing ChatGPT*. (2022, November 30). OpenAI. <https://openai.com/index/chatgpt/>
- Liu, Z., Hu, H., Lin, Y., Yao, Z., Yixuan, Z., Jia, W., Yue, N., Zheng, C., Li, Z., Furu, D., & Guo, W. (2022). *Swin Transformer V2: Scaling Up Capacity and Resolution*. <https://arxiv.org/pdf/2111.09883>
- Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). *A ConvNet for the 2020s*. arXiv. <https://arxiv.org/abs/2201.03545>
- Mapillary AB. (n.d.). *API documentation*. Mapillary. Retrieved December 2, 2025, from <https://www.mapillary.com/developer/api-documentation>
- Mapillary AB. (2013). *Mapillary* [Web service]. <https://www.mapillary.com>
- McIntyre, D., Rebel, S., Bali, S. (2025). *Canadian\_streetview\_cities\_models* [Computer software]. Hugging Face. [https://huggingface.co/3106Project/canadian\\_streetview\\_cities\\_models](https://huggingface.co/3106Project/canadian_streetview_cities_models)
- Natural Resources Canada. (2022, December 16). *Geolocation service*. Government of Canada. <https://natural-resources.canada.ca/maps-tools-publications/satellite-elevation-air-photos/geolocation-service>
- PyTorch. (n.d.). *Transforms*. Retrieved December 2, 2025, from <https://pytorch.org/vision/stable/transforms.html>
- Rebel, S., McIntyre, D., & Bali, S. (2025). *Canadian-streetview-cities* [Data set]. Hugging Face. <https://huggingface.co/datasets/SABR22/Canadian-streetview-cities>
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., & Fei-Fei, L. (2015). *ImageNet large scale visual recognition challenge*. arXiv. <https://arxiv.org/abs/1409.0575>
- SmugMug, Inc. (2004). Flickr [Web service]. <https://www.flickr.com>
- Suresh, S., Chodosh, N., & Abello, M. (2018). *DeepGeo: Photo localization with deep neural network*. arXiv. <https://arxiv.org/abs/1810.03077>
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2014). *Going deeper with convolutions*. arXiv. <https://arxiv.org/abs/1409.4842>
- Tan, M., & Le, Q. V. (2020). *EfficientNet: Rethinking model scaling for convolutional neural networks*. arXiv. <https://arxiv.org/abs/1905.11946>

- Tan, M., & Le, Q. V. (2021). *EfficientNetV2: Smaller models and faster training*. arXiv. <https://arxiv.org/abs/2104.00298>
- Touvron, H., Cord, M., & Hervé Jégou. (2022). *DeiT III: Revenge of the ViT*. <https://doi.org/10.48550/arxiv.2204.07118>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. <https://arxiv.org/pdf/1706.03762>
- Vo, N. N., Jacobs, N., & Hays, J. (2017). *Revisiting IM2GPS in the deep learning era*. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2640–2649. <https://doi.org/10.1109/ICCV.2017.286>
- Weyand, T., Kostrikov, I., & Philbin, J. (2016). PlaNet – *Photo geolocation with convolutional neural networks*. In B. Leibe, J. Matas, N. Sebe, & M. Welling (Eds.), *Computer Vision – ECCV 2016* (pp. 37–55). Springer. [https://doi.org/10.1007/978-3-319-46484-8\\_3](https://doi.org/10.1007/978-3-319-46484-8_3)
- Wightman, R. (2019). *PyTorch image models* [Computer software]. GitHub. <https://zenodo.org/records/7618837>
- Xie, Y., Li, Y., Zhang, J., Zhang, J., & Kuang, Z. (2024). *ANALYSIS OF DIFFERENCES IN STREET VISUAL WALKABILITY PERCEPTION BETWEEN DCNN AND VIT MODEL BASED ON PANORAMIC STREET VIEW IMAGES*. <https://caadria2024.org/wp-content/uploads/2024/04/449-ANALYSIS-OF-DIFFERENCES-IN-STREET-VISUAL-WALKABILITY-PERCEPTION-BETWEEN-DCNN-AND-VIT-MODEL-BASED-ON-PANORAMIC-STREET-VIEW-IMAGES.pdf>

## Link to Github Classroom Implementation

Find a copy of the full implementation from the Group 73 project repository here: <https://github.com/StephenRebel/canada-guesser>